

SOLUCION

Actividad 1 — Construcción manual del BST

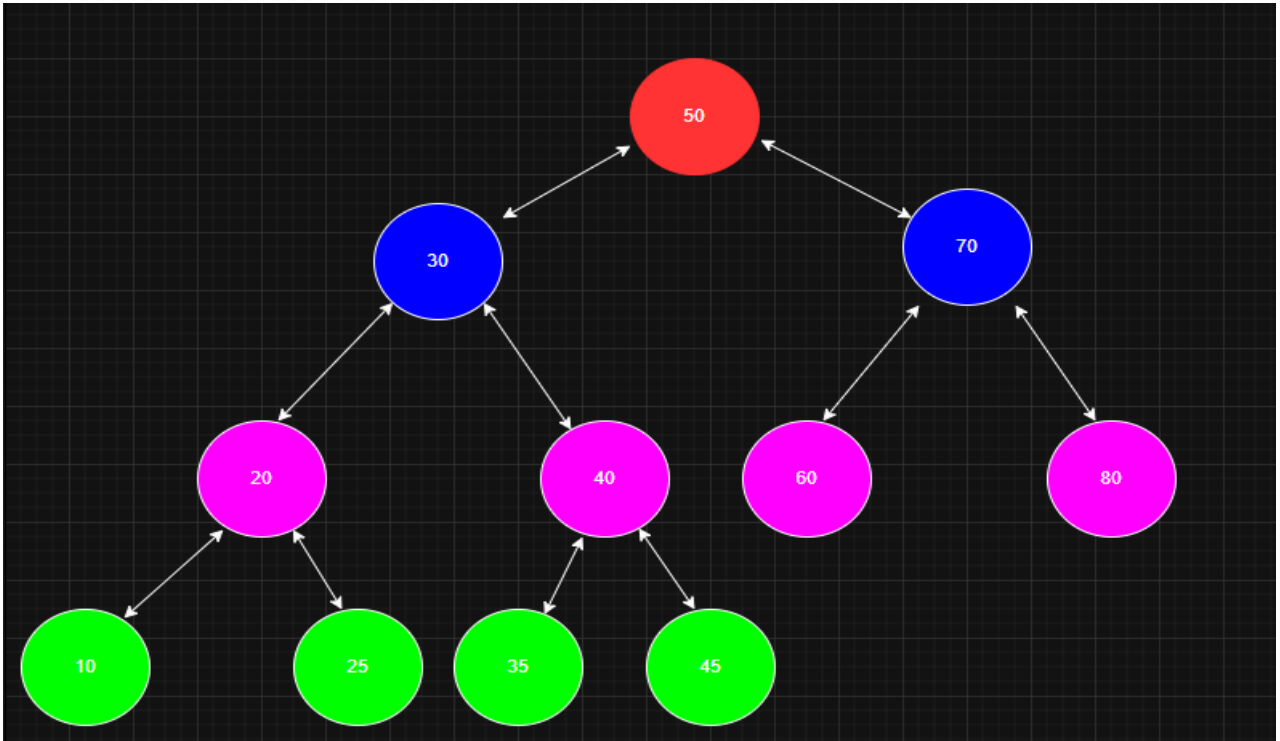


TABLA DE DECISIONES

Código Comparaciones

Ubicación final

50	El árbol estaba vacío	Quedó como raíz
30	30 es menor que 50	Se insertó a la izquierda de 50
70	70 es mayor que 50	Se insertó a la derecha de 50
20	20 es menor que 50 y menor que 30	Se insertó a la izquierda de 30
40	40 es menor que 50 pero mayor que 30	Se insertó a la derecha de 30

Actividad 2 — Recorridos

Recorrido PreOrden — Salida esperada: 50, 30, 20, 10, 25, 40, 35, 45, 70, 60, 80

Recorrido InOrden — Salida esperada: 10, 20, 25, 30, 35, 40, 45, 50, 60, 70, 80

Recorrido PostOrden — Salida esperada: 10, 25, 20, 35, 45, 40, 30, 60, 80, 70, 50

- indique cuál de los tres entregaría usted a FEDEGÁN si le pidieran un “listado oficial ordenado de fincas”

El mejor recorrido sería el **InOrden** porque en un BST este recorrido siempre muestra los códigos en orden ascendente ósea de forma en que siempre sube y pues ayuda a generar los reportes mas organizados y pues también facilitaría la búsqueda de registros

Actividad 3 — Implementación en Java

```
public class NodoBST {  
  
    int codigo;  
    String nombreFinca;  
    String municipio;  
    NodoBST izquierdo;  
    NodoBST derecho;  
  
    public NodoBST(int codigo, String nombreFinca, String municipio) {  
        this.codigo = codigo;  
        this.nombreFinca = nombreFinca;  
        this.municipio = municipio;  
        this.izquierdo = null;  
        this.derecho = null;  
    }  
}
```

```

public class ArbolBST {
    private NodoBST raiz;
    public ArbolBST() {
        raiz = null;
    }
    public void insertar(int codigo, String nombre, String municipio) {
        raiz = insertarRec(raiz, codigo, nombre, municipio);
    }
    private NodoBST insertarRec(NodoBST actual, int codigo, String nombre, String municipio) {
        if (actual == null) {
            return new NodoBST(codigo, nombre, municipio);
        }
        if (codigo < actual.codigo) {
            actual.izquierdo = insertarRec(actual.izquierdo, codigo, nombre, municipio);
        } else if (codigo > actual.codigo) {
            actual.derecho = insertarRec(actual.derecho, codigo, nombre, municipio);
        }
        return actual;
    }
    public NodoBST buscar(int codigo) {
        return buscarRec(raiz, codigo);
    }
    private NodoBST buscarRec(NodoBST actual, int codigo) {
        if (actual == null || actual.codigo == codigo) {
            return actual;
        }
        if (codigo < actual.codigo) {
            return buscarRec(actual.izquierdo, codigo);
        }
        return buscarRec(actual.derecho, codigo);
    }
    public void preOrden() {
        preOrdenRec(raiz);
        System.out.println();
    }
    private void preOrdenRec(NodoBST nodo) {
        if (nodo != null) {
            System.out.print(nodo.codigo + " ");
            preOrdenRec(nodo.izquierdo);
            preOrdenRec(nodo.derecho);
        }
    }
    public void inOrden() {
        inOrdenRec(raiz);
        System.out.println();
    }
    private void inOrdenRec(NodoBST nodo) {
        if (nodo != null) {
            inOrdenRec(nodo.izquierdo);
            System.out.print(nodo.codigo + " ");
            inOrdenRec(nodo.derecho);
        }
    }
    public void postOrden() {
        postOrdenRec(raiz);
        System.out.println();
    }
    private void postOrdenRec(NodoBST nodo) {
        if (nodo != null) {
            postOrdenRec(nodo.izquierdo);
            postOrdenRec(nodo.derecho);
            System.out.print(nodo.codigo + " ");
        }
    }
}

```

```

public class Main {

    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        ArbolBST arbol = new ArbolBST();

        arbol.insertar(codigo: 50, nombre: "Hato La Aurora", municipio: "Hato Corozal");
        arbol.insertar(codigo: 30, nombre: "Finca El Encanto", municipio: "Yopal");
        arbol.insertar(codigo: 70, nombre: "Hacienda La Esperanza", municipio: "Aguazul");
        arbol.insertar(codigo: 20, nombre: "Finca San Isidro", municipio: "Nunchía");
        arbol.insertar(codigo: 40, nombre: "Hato Caribayona", municipio: "Paz de Ariporo");
        arbol.insertar(codigo: 60, nombre: "Finca Las Brisas", municipio: "Tauramena");
        arbol.insertar(codigo: 80, nombre: "Hacienda La Palma", municipio: "Maní");
        arbol.insertar(codigo: 10, nombre: "Finca El Porvenir", municipio: "Pore");
        arbol.insertar(codigo: 25, nombre: "Finca Las Garzas", municipio: "Trinidad");
        arbol.insertar(codigo: 35, nombre: "Hato La Cabaña", municipio: "Orocué");
        arbol.insertar(codigo: 45, nombre: "Finca Buenavista", municipio: "Monterrey");

        System.out.println(x: "Recorrido PreOrden:");
        arbol.preOrden();

        System.out.println(x: "Recorrido InOrden:");
        arbol.inOrden();

        System.out.println(x: "Recorrido PostOrden:");
        arbol.postOrden();

        NodoBST encontrado = arbol.buscar(codigo: 45);

        if (encontrado != null) {
            System.out.println("Finca encontrada: "
                + encontrado.nombreFinca);
        } else {
            System.out.println(x: "No encontrada");
        }
    }
}

```

```

Recorrido PreOrden:
50 30 20 10 25 40 35 45 70 60 80
Recorrido InOrden:
10 20 25 30 35 40 45 50 60 70 80
Recorrido PostOrden:
10 25 20 35 45 40 30 60 80 70 50
Finca encontrada: Finca Buenavista
PS C:\proyecto Maven>

```

Actividad 4 — Análisis y reflexión

1. ¿Cuál es la altura del árbol resultante?

La altura es de 4 niveles ya que 50,30,20,10 es el camino que es mas largo desde la raíz asta una hoja

2. ¿Cuántas comparaciones se requieren para buscar la finca con código 45? ¿Y para una finca que no existe, por ejemplo el código 33?

Se requieren 4 comparaciones para encontrar el 45

Se requiere 4 comparaciones se compara con 50 con 30 con 40 y con 35 después intenta seguir hacia la izquierda pero encuentra un espacio vacío concluyendo que no existe

3. Si las fincas se hubieran insertado en orden ascendente (10, 20, 25, 30, 35...), ¿cómo se vería el árbol? ¿Qué complejidad tendría la búsqueda?

el árbol quedaría desbalanceado porque cada nuevo número sería mayor que el anterior y siempre se colocaría hacia la derecha

La búsqueda tendría una complejidad lineal porque habría que revisar los nodos uno por uno casi como si fuera una lista, y no un árbol balanceado

4. ¿En qué situación real recomendaría usar PostOrden en este sistema? Dé un ejemplo concreto del contexto ganadero.

El recorrido PostOrden sería útil cuando se necesite eliminar información del árbol sin causar errores este recorrido primero elimina los nodos hijos y al final el nodo padre, lo que ayuda a liberar correctamente la memoria un ejemplo en el contexto ganadero sería cuando FEDEGÁN necesite borrar todos los registros temporales de las fincas después de terminar una jornada de vacunación o una campaña sanitaria de esta manera primero se eliminan las fincas registradas y al final la raíz principal del árbol